Robótica Inteligente: Práctica I

Introducción a las herramientas básicas para robótica móvil y manipulación: ROS2, Gazebo, Navigation Stack, MoveIt y datasets públicos



Dpto. de Física, Ingeniería de Sistemas y Teoría de la Señal

M.A. Muñoz-Bañón - Curso Académico 2026/2027

Índice

1. Introducción y Configuración del Entorno Docker **3**
1.1. Introducción . . . 3
1.2. El Entorno de Trabajo Basado en Docker . . . 3

2. Bloque 1 - Fundamentos de ROS 2 **5**
2.1. Objetivos del Bloque . . . 5
2.2. Conceptos Básicos . . . 6
2.3. Tutoriales Oficiales de ROS 2 Jazzy . . . 6
2.4. Referencia Rápida de Comandos de Consulta (Cheat Sheet) . . . 6

3. Bloque 2 - Mapeo y Navegación Autónoma (Nav2) **7**
3.1. Contexto y Objetivos del Bloque . . . 7
3.2. Guía de Ejecución y Control Interactivo . . . 7
3.2.1. Fase A: Construcción y Guardado del Mapa (SLAM) . . . 8
3.2.2. Fase B: Despliegue de la Navegación (Modularidad Nav2) . . . 9

4. Bloque 3 - Odometría LiDAR-Inercial (LIO) con Datos Reales **11**
4.1. Contexto y Objetivos del Bloque . . . 11
4.2. Guía de Ejecución de RKO-LIO con Oxford Spires Dataset . . . 12

5. Bloque 4 - Manipulación Robótica (MoveIt 2) **13**
5.1. Contexto y Objetivos del Bloque . . . 13
5.2. Guía de Ejecución y Control Interactivo . . . 13

6. Entrega y Criterios de Evaluación **15**
6.1. Estructura de la Memoria Técnica Obligatoria . . . 15
6.2. Tareas de Programación (Opcionales para Subir Nota) . . . 15
6.3. Criterios de Evaluación . . . 16

1. Introducción y Configuración del Entorno Docker

1.1. Introducción

El objetivo fundamental de esta práctica es que el alumnado pueda familiarizarse con las herramientas de software utilizadas en robótica, como lo son el middleware estándar en investigación e industria **ROS 2 (Jazzy Jalisco)** y el simulador de física avanzada **Gazebo Harmonic**. En dicho simulador, se utilizarán algoritmos de localización y planificación, tanto para robots móviles como para manipuladores. Además, en localización, se utilizarán también algoritmos avanzados con los que se utilizarán datos reales.

Para esta práctica se proporcionará software ya desarrollado que el alumnado deberá ejecutar y analizar para comprender el funcionamiento y la estructura del mismo. También se pedirá desarrollar scripts sencillos para interactuar con dicho software. Durante el desarrollo de la práctica, el profesorado realizará las preguntas pertinentes para comprobar el resultado del aprendizaje (Sección 6).

La práctica está dividida en cuatro bloques secuenciales:

1. **Fundamentos de ROS 2:** Familiarización con la arquitectura de nodos, tópicos, servicios y acciones. Para esto se realizarán los tutoriales que se proporcionan en la propia web.
2. **Navegación Autónoma (Nav2):** Mapeado de entornos utilizando la herramienta *SLAM Toolbox* y configuración de la pila de navegación autónoma y evasión de obstáculos utilizando el stack *Nav2*.
3. **Odometría LiDAR-Inercial (LIO):** Procesamiento de datos sensoriales reales y densos utilizando el *Oxford Spires Dataset* sobre un pipeline LIO para adquirir experiencia más allá de las simulaciones.
4. **Manipulación Robótica (MoveIt 2):** Integración de un brazo robótico industrial Universal Robots UR3e y planificación cinemática en el espacio cartesiano.

1.2. El Entorno de Trabajo Basado en Docker

Para garantizar la reproducibilidad de los experimentos y mitigar los conflictos de dependencias de software, toda la suite de desarrollo se encuentra empaquetada en una imagen de **Docker**. Cuando se lanza el contenedor, se configura con persistencia de datos y soporte de aceleración gráfica por hardware (compatible con GPUs dedicadas), lo que garantiza un rendimiento fluido de las interfaces *RViz2* y *Gazebo*.

A diferencia de los entornos efímeros tradicionales, utilizaremos un contenedor persistente. Esto permite que las herramientas adicionales que instale el alumno o las modificaciones del sistema se conserven entre sesiones, manteniendo el código y los mapas a salvo en el sistema anfitrión mientras el contenedor no se cierre.

Paso 1: Creación de Imagen y del Espacio de Trabajo en el Host: La imagen *Docker* se encuentra ya instalada en los PCs de las aulas de prácticas. Para ver las imágenes instaladas, ejecutar el comando: `docker image ls`. La imagen que se utilizará durante el curso es: `robotica_inteligente_grado`. Se recomienda encarecidamente trabajar en la imagen del laboratorio, ya que en el laptop personal del alumnado pueden surgir imprevistos técnicos que retrasen el desarrollo de las tareas. Sin embargo, si a pesar de esto alguien quiere instalarla en su ordenador personal, deberá descargar la *Dockerfile* y ejecutar el siguiente comando en el directorio donde se encuentre el archivo:

```
# Construccion de la imagen docker proporcionada
docker build -t robotica_inteligente_grado:latest .
```

Antes de lanzar el contenedor, el alumnado debe asegurarse de que existe el directorio de trabajo en la máquina física. Este directorio se sincronizará en tiempo real con el contenedor.

```
# Ejecutar en la terminal de la maquina fisica (Anfitrion)
mkdir -p ~/ros2_ws
```

Paso 2: Despliegue Inicial del Contenedor A partir de la imagen instalada en el paso anterior, se podrá construir el contenedor *Docker* con los comandos especificados más abajo. Los flags añadidos al comando sirven, entre otras cosas, para mapear la carpeta compartida entre el contenedor y el host, activar la aceleración gráfica de NVIDIA y permitir que las ventanas del contenedor se dibujen en la pantalla del host:

```
# Permitir conexiones al servidor de ventanas X11
xhost +local:docker

# Crear y arrancar el contenedor definitivo
docker run -it \
    --name robotica_inteligente_grado \
    --user developer \
    --network host \
    --ipc host \
    --pid host \
    --security-opt seccomp=unconfined \
    --env DISPLAY=$DISPLAY \
    --env QT_X11_NO_MITSHM=1 \
    --env NVIDIA_VISIBLE_DEVICES=all \
    --env NVIDIA_DRIVER_CAPABILITIES=all \
    --env __NV_PRIME_RENDER_OFFLOAD=1 \
    --env __GLX_VENDOR_LIBRARY_NAME=nvidia \
    --env __GL_THREADED_OPTIMIZATIONS=1 \
    --volume /tmp/.X11-unix:/tmp/.X11-unix:rw \
    --volume ~/ros2_ws:/home/developer/ros2_ws:rw \
    --device /dev/dri \
    --gpus all \
    robotica_inteligente_grado:latest
```

A continuación se detalla la función de los flags críticos utilizados en el comando superior:

`-it` Activa el modo interactivo (`-i`) y asigna una pseudo-TTY (`-t`), permitiendo interactuar con la terminal del contenedor en tiempo real.

`-name robotica_inteligente_grado` Asigna un nombre estático al contenedor. Esto evita crear contenedores duplicados y permite despertarlo en futuras sesiones.

`-user developer` Fuerza la ejecución con el usuario sin privilegios del contenedor, evitando que los archivos creados en el espacio de trabajo pertenezcan a *root*.

`-network host` Comparte la pila de red del sistema anfitrión. En ROS 2, esto permite que el descubrimiento de nodos por DDS funcione de forma nativa sin redirigir puertos.

`-ipc host & -pid host` Comparten la memoria inter-procesos y la tabla de IDs de procesos del anfitrión. Es fundamental para habilitar la comunicación de "zero-copy" de alta velocidad entre nodos de ROS 2.

`-security-opt seccomp=unconfined` Desactiva el perfil de seguridad que intercepta las llamadas al sistema (*syscalls*). Vital para eliminar cuellos de botella de CPU al procesar algoritmos pesados como odometrías LIO o SLAM.

`-env DISPLAY=$DISPLAY & -volume /tmp/.X11-unix...` Redirigen e inyectan el socket gráfico de tu Ubuntu físico para permitir que el contenedor dibuje las interfaces de RViz2 y Gazebo.

`-volume ~/ros2_ws...` Crea el volumen compartido (mapeo) entre tu disco duro real y el contenedor. Todo lo que se guarde aquí sobrevivirá al apagado del sistema.

`-gpus all & -runtime nvidia & -env __NV_...` Configuran el paso directo de hardware hacia la GPU NVIDIA y activan las variables de *PRIME Offload* para exprimir la aceleración 3D en portátiles híbridos o PCs de escritorio.

Paso 3: Recuperación del Entorno (Siguientes Sesiones) Dado que el comando anterior no incluye el flag `-rm`, al cerrar la terminal el contenedor pasará al estado *Stopped*, pero no se destruirá. Para verificar si el contenedor está activo, se puede ejecutar en un terminal en la máquina física (anfitrión) el comando: `docker ps -a`. Para retomar el trabajo en las siguientes sesiones de laboratorio o abrir nuevas terminales de trabajo, el alumnado utilizará exclusivamente el siguiente flujo:

```
# 1. Volver a encender el contenedor latente (Si se apago el PC)
docker start robotica_inteligente_grado

# 2. Entrar de forma interactiva con el usuario de desarrollo
docker exec -it --user developer robotica_inteligente_grado bash
```

Nota: Cada vez que se necesite una nueva pestaña en la terminal para un nodo diferente (por ejemplo, para separar Gazebo de Nav2), bastará con abrir otra pestaña en el host y repetir el comando `docker exec` indicado arriba.

Si en algún momento se desea cerrar el contenedor (con lo que se **perdería TODO lo que se haya instalado sobre el contenedor original** excepto lo que se encuentre en la carpeta compartida), se ejecutará el siguiente comando:

```
# Cerrar el contenedor
docker rm -f robotica_inteligente_grado
```

2. Bloque 1 - Fundamentos de ROS 2

2.1. Objetivos del Bloque

Este primer bloque de la práctica tiene como finalidad sentar las bases operativas del ecosistema **ROS 2 (Robot Operating System)**. El alumnado aprenderá a romper con la arquitectura de software monolítica tradicional, entendiendo cómo un sistema robótico se compone de múltiples procesos especializados independientes (nodos) que se comunican entre sí de forma distribuida y asíncrona. Para ello, se deberán realizar los tutoriales que se proponen en la propia documentación de ROS 2.

Al finalizar este bloque, el estudiante deberá dominar el uso de la terminal para interrogar al sistema, monitorizar el flujo de datos en tiempo real y diagnosticar el estado del grafo de conexiones entre nodos.

2.2. Conceptos Básicos

Durante la realización de los tutoriales, se debe prestar especial atención a algunos conceptos clave, que será necesario comprender y dominar para la realización de los siguientes bloques. Esos conceptos son los siguientes:

- **Nodos (*Nodes*):** Entidades individuales (ejecución de un programa) encargadas de una tarea de cómputo específica (ej., leer un sensor, controlar un motor, etc.).
- **Tópicos (*Topics*):** Canales de comunicación unidireccionales basados en un modelo de publicación/suscripción (*Publish/Subscribe*), ideales para flujos de datos continuos (odometría, nubes de puntos, etc.).
- **Mensajes (*Interfaces/Msgs*):** Estructuras de datos fuertemente tipadas que definen la información que viaja por los tópicos.
- **Servicios (*Services*):** Esquema de comunicación síncrono basado en petición/respuesta (*Request/Response*) para acciones puntuales o configuraciones.
- **Acciones (*Actions*):** Tipo de comunicación asíncrona para tareas de larga duración que requieren seguimiento. Consta de tres partes: una meta (*Goal*), feedback periódico y un resultado final (esencial en *Nav2* y *MoveIt 2*).
- **Parámetros (*Parameters*):** Variables de configuración específicas de cada nodo que pueden inicializarse por fichero y/o modificarse en tiempo de ejecución.
- **Transformaciones (*Tf2*):** Es lo que define las transformaciones entre sistemas de referencia. Por ejemplo, la transformación entre la base de un robot y un sensor.

2.3. Tutoriales Oficiales de ROS 2 Jazzy

Para adquirir la soltura necesaria, el alumnado deberá acceder a la documentación oficial de ROS 2 Jazzy Jalisco a través del siguiente enlace:

https://docs.ros.org/en/jazzy/index.html

En la documentación prestaremos especial atención a la sección **Tutoriales**, donde estos se encuentran divididos en 3 niveles de dificultad. Solamente se requiere realizar los relacionados con el nivel *Beginner*, así como los relacionados con *Tf2*. También prestar atención a la sección **Concepts**, donde se explican los conceptos que se prueban de manera práctica en los tutoriales.

2.4. Referencia Rápida de Comandos de Consulta (Cheat Sheet)

Complementariamente a la terminal, el alumnado debe aprender a lanzar la herramienta de introspección gráfica para visualizar la topología de la red de nodos. Por ejemplo, se puede ejecutar el siguiente comando dentro del contenedor cuando tenga procesos corriendo para analizar las conexiones:

```
# Lanzar el analizador grafico del grafo de nodos
ros2 run rqt_graph rqt_graph
```

A modo de consulta rápida durante las sesiones de laboratorio, el alumnado puede apoyarse en la siguiente tabla de herramientas CLI esenciales:

Comando Base	Descripción / Función
`ros2 node list`	Lista todos los nodos que se están ejecutando en el grafo actual.
`ros2 node info /nombre_nodo`	Muestra los tópicos, servicios y acciones asociados a ese nodo.
`ros2 topic list`	Muestra todos los tópicos activos en el sistema.
`ros2 topic info /nombre_topico`	Muestra información de tipo de mensaje, así como de conexiones con nodos.
`ros2 topic echo /nombre_topico`	Vuelca en la pantalla los mensajes del tópico en tiempo real.
`ros2 topic hz /nombre_topico`	Calcula la frecuencia media de publicación de un tópico.
`ros2 service list`	Lista todos los servicios que ofrecen respuesta en el entorno.
`ros2 action list`	Lista las acciones activas disponibles (ej. ;/move_action).
`ros2 param list`	Muestra las variables de configuración disponibles en los nodos.

Tabla 1: Comandos esenciales de inspección en ROS 2.

3. Bloque 2 - Mapeo y Navegación Autónoma (Nav2)

3.1. Contexto y Objetivos del Bloque

La navegación autónoma permite a un robot móvil desplazarse por un entorno desconocido o previamente mapeado de forma lo más segura y eficiente posible. En esta sección nos centraremos en la navegación 2D, es decir, la navegación que aborda la cinemática de bases móviles restringidas al plano $\mathbb{SE}(2)$, teniendo que lidiar con incertidumbres sensoriales, mapas probabilísticos y obstáculos dinámicos.

En este bloque se utilizará la plataforma robótica diferencial **TurtleBot 4** en el simulador **Gazebo**. El alumnado experimentará con los dos grandes paradigmas de la robótica móvil mediante el framework **Nav2**:

1. **Fase de SLAM (*Simultaneous Localization and Mapping*):** El robot construye un mapa bidimensional de ocupación del entorno mediante el algoritmo *SLAM Toolbox* mientras estima su propia posición utilizando lecturas de odometría y un sensor LiDAR 2D.

2. **Fase de Navegación Autónoma:** Con un mapa estático ya consolidado, el robot se localiza probabilísticamente mediante un filtro de partículas (*AMCL*). El motor de Nav2 utiliza mapas de coste (*Costmaps*) divididos en capas (estática, obstáculos e inflación) para alimentar de forma paralela a un planificador global (algoritmo basado en celdas de red como *Navfn*) y a un controlador local dinámico predictivo (*MPPI Controller*) para gobernar los motores evitando colisiones.

3.2. Guía de Ejecución y Control Interactivo

La imagen que se ha proporcionado y que se encuentra instalada en los ordenadores de los laboratorios contiene el simulador **Gazebo** con el robot **TurtleBot 4**. Para ejecutarlo hay que entrar en el contenedor *Docker* y ejecutar el siguiente comando:

```
# Terminal 1: Lanzar el entorno virtual con el algoritmo SLAM activo
ros2 launch turtlebot4_gz_bringup turtlebot4_gz.launch.py
```

Tras lanzar el simulador, se abrirá una ventana como la que se muestra en la Figura 1, en la que, por defecto, se ha lanzado el entorno *Warehouse*. Para controlar el robot con las teclas del ordenador, basta con seleccionar la pestaña *keyboard* en la parte inferior del cuadro de mandos de la parte derecha.

Una vez lanzada la simulación, las tareas para desarrollar este bloque se dividen secuencialmente en dos fases operativas: la generación del mapa físico y el despliegue del stack de navegación autónoma.



Figura 1: Vista del simulador **Gazebo** en el entorno *Warehouse* con el robot *TurtleBot 4*.

3.2.1. Fase A: Construcción y Guardado del Mapa (SLAM)

Para lanzar el algoritmo con el que se realizará el mapeado del entorno (SLAM), en una segunda terminal se debe ejecutar el siguiente comando:

```
# Terminal 2: Lanzar el algoritmo SLAM
ros2 launch turtlebot4_navigation slam.launch.py use_sim_time:=true
```

Para visualizar el mapa que se está generando, se debe abrir la aplicación de ROS2 *Rviz2*. Para ello, en una tercera terminal se ejecutará el siguiente comando:

```
# Terminal 3: Lanzar Rviz2 para visualizar el mapa y el robot
ros2 launch turtlebot4_viz view_navigation.launch.py use_sim_time:=true
```

En *Rviz2* aparecerá el mapa generado hasta ahora, que consta únicamente de lo que el robot ve en su posición inicial (Figura 2).

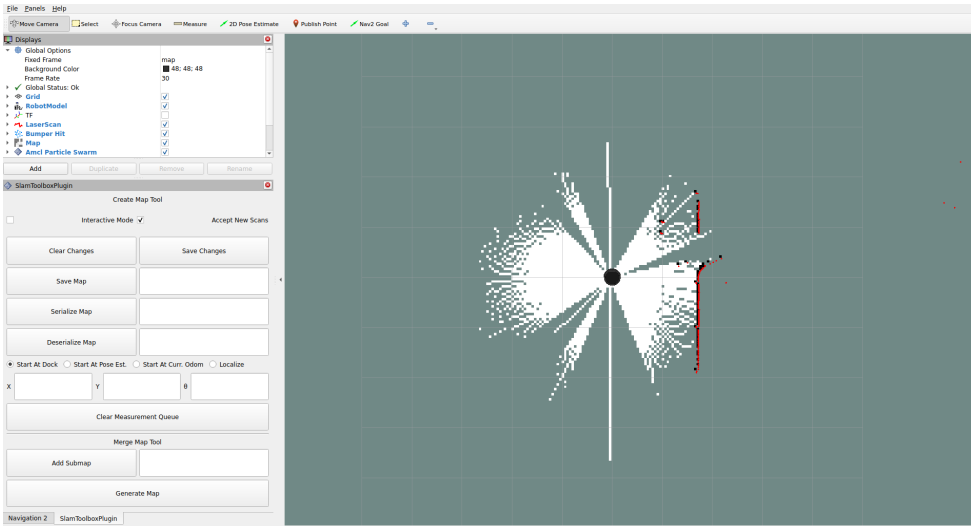


Figura 2: Vista de *Rviz2* donde se muestra el mapa inicial.

Ahora, mediante el teclado, se debe conducir el robot de forma remota por los pasillos. Se podrá ir comprobando en *RViz2* cómo el mapa va creciendo.

Una vez que el mapa realizado sea nítido y cubra la zona de interés, se podrá guardar el mapa utilizando el botón `Save Map`. Alternativamente, se puede abrir otra terminal en el contenedor y guardar el mapa utilizando el servidor de mapas:

```
# Terminal 4: Guardar permanentemente el mapa generado
ros2 run nav2_map_server map_saver_cli -f /home/developer/ros2_ws/
    mapa_laboratorio
```

Esto generará dos archivos críticos en el espacio de trabajo: `mapa_laboratorio.pgm` (la imagen de ocupación, que se puede visualizar con un visor de imagen cualquiera) y `mapa_laboratorio.yaml` (los metadatos de resolución y origen). Estos archivos representan el mapa generado del entorno, y esto es lo que se utilizará para localizar el robot en la fase de navegación autónoma.

3.2.2. Fase B: Despliegue de la Navegación (Modularidad Nav2)

Antes de arrancar el proceso de navegación autónoma, hay que asegurarse de cerrar los nodos previamente utilizados. Para arrancar el proceso de navegación autónoma se realizarán cuatro pasos que se ejecutarán en tres terminales. Los pasos a realizar son los siguientes:

Paso 1: Servidor de Localización y Mapas

Cargar el mapa guardado. Este comando inicializa el `map_server` y el nodo de localización llamado `amcl`:

```
# Terminal 1: Lanzar la infraestructura de mapa estatico y AMCL
ros2 launch turtlebot4_navigation localization.launch.py map:=/home/
    developer/ros2_ws/mapa_laboratorio.yaml use_sim_time:=true
```

Paso 2: Motores de Navegación (Nav2)

En una segunda terminal, activar los planificadores, controladores de trayectoria y árboles de comportamiento. Estos nodos se suscribirán al tópico `/map` publicado por la Terminal 1 de forma transparente:

```
# Terminal 2: Arrancar los servidores de planificacion y control de Nav2
ros2 launch turtlebot4_navigation nav2.launch.py use_sim_time:=true
```

Paso 3: Interfaz Gráfica de Usuario

Abrir una tercera terminal para visualizar el sistema completo en RViz2:

```
# Terminal 3: Lanzar la interfaz de visualizacion
ros2 launch turtlebot4_viz view_navigation.launch.py use_sim_time:=true
```

En *RViz2* se podrá visualizar el mapa que se ha cargado, y que se corresponde con el realizado en la Fase A (Figura 3).

Paso 4: Sincronización e Interacción en RViz2 Una vez que las tres terminales estén estables y se vea el mapa correctamente en *RViz2*, se procederá al guiado interactivo:

1. El robot aparecerá desalineado de forma inicial. Para alinearlo con el mapa, en la barra superior de RViz2, pulsar el botón **2D Pose Estimate**.

2. Hacer clic en el mapa sobre la ubicación estimada donde se observe al robot dentro de Gazebo y **arrastrar el ratón** para definir la orientación de su morro. El filtro de partículas AMCL se alineará dispersando una nube de flechas verdes que colapsará sobre el robot.

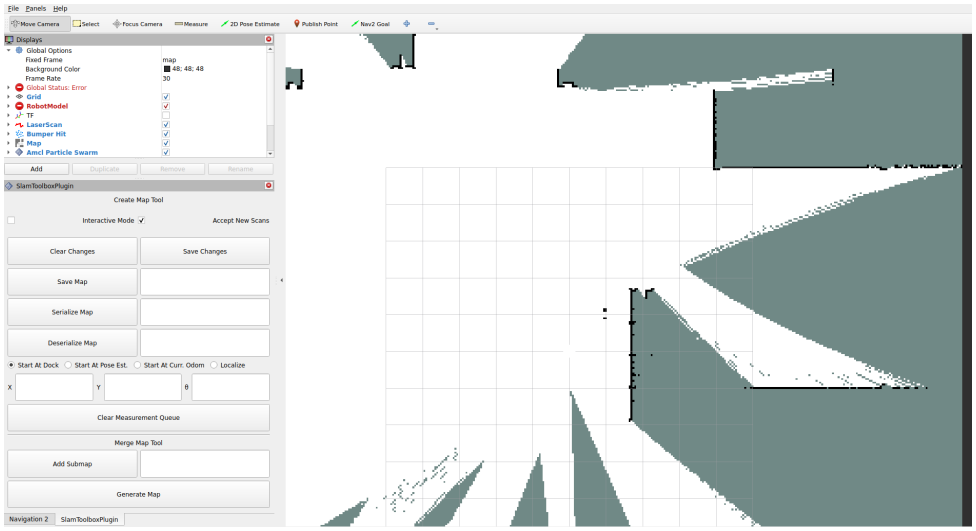


Figura 3: Visualización del mapa realizado en la Fase A y cargado en la Fase B con *AMCL*.

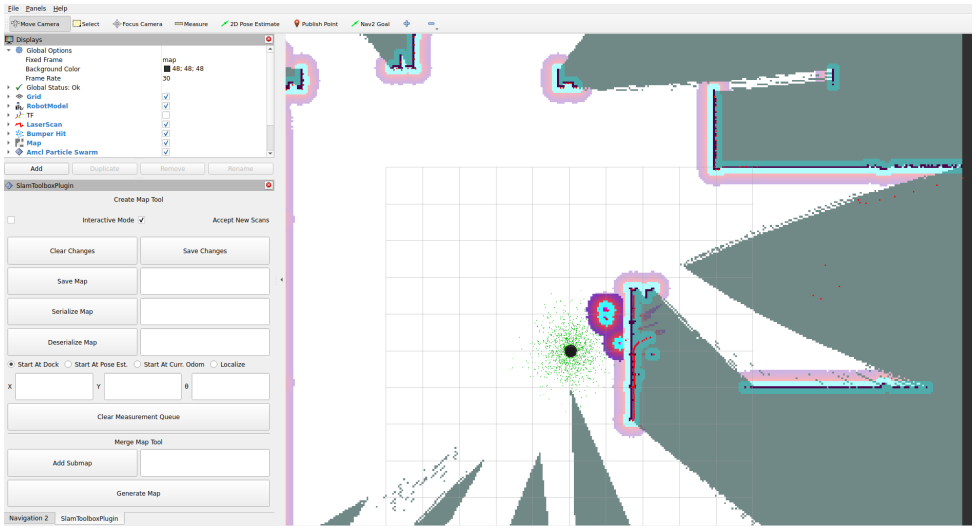


Figura 4: Una vez localizado el robot, se puede ver el mapa de coste.

3. Observar cómo brotan las zonas coloreadas sobre los obstáculos (los mapas de coste local y global reflejan el rango de inflación de seguridad). En la Figura 4, se muestra un ejemplo.

4. Para arrancar una navegación autónoma, seleccionar el botón **2D Goal Pose** de la barra superior, pinchar en cualquier zona libre del mapa y marcar la orientación de destino. De esta forma se le envía la pose deseada o *goal*. En este momento aparecerá en rojo la trayectoria que el robot va a seguir y empezará a moverse de forma autónoma siguiéndola. En la Figura 5 se muestra esta ruta estimada y al robot navegando por ella.

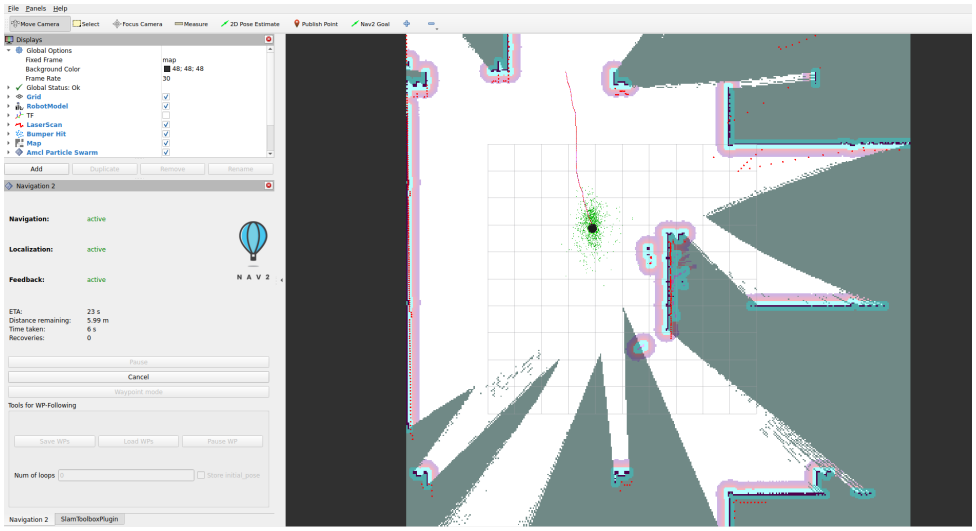


Figura 5: Ejemplo de ruta calculada hasta el destino enviado manualmente.

4. Bloque 3 - Odometría LiDAR-Inercial (LIO) con Datos Reales

4.1. Contexto y Objetivos del Bloque

Hasta este punto, el alumnado ha trabajado exclusivamente en entornos de simulación controlados donde la física es ideal y el tiempo de simulación se detiene si el procesador se satura, lo cual no ocurre cuando operamos en el mundo real. En este tercer bloque de la práctica, se introduce el desafío de procesar datos sensoriales masivos procedentes de sensores operando en el mundo real utilizando el framework de odometría LiDAR-Inercial (LIO) llamado ***rko_lio***, que se encuentra instalado en la imagen *Docker* proporcionada.

Los algoritmos LIO estiman la trayectoria de un vehículo combinando la alta frecuencia y la respuesta rápida de una Unidad de Medición Inercial (IMU, ∼200 Hz) con la información estructural geométrica de un sensor LiDAR 3D (∼10-20 Hz). La IMU proporciona una estimación rápida de la aceleración y la velocidad angular a corto plazo, mientras que el LiDAR corrige la deriva métrica acumulada (*drift*) mediante el emparejamiento iterativo de nubes de puntos (*Scan-to-Map Matching*).

Los datos de sensores para alimentar este método LIO se reproducirán de manera *offline* desde el extensamente utilizado en robótica: ***Oxford Spires Dataset* (OSD)**:

https://dynamic.robots.ox.ac.uk/datasets/oxford-spires/,

un banco de datos (*dataset*) capturado en un entorno urbano real denso. Este escenario rompe los paradigmas de la simulación al introducir:

- **Densidad Sensorial:** Nubes de puntos de decenas de miles de coordenadas tridimensionales por segundo, lo que genera una carga de cómputo masiva en el procesador.
- **Fusión de Datos Descentralizada:** Sincronización estricta por marcas de tiempo (*timestamps*) de ROS 2 entre la IMU y el LiDAR.
- **Sensibilidad Temporal:** El algoritmo requiere que la diferencia de tiempo (*delta*) entre escaneos LiDAR sea constante para que las ecuaciones de preintegración de la IMU no diverjan. Si el hilo de procesamiento de la CPU se satura (un cuello de botella común en entornos virtualizados), el sistema acumula retraso, superando el umbral crítico de seguridad y abortando el procesamiento de los fotogramas (*dropping frames*).

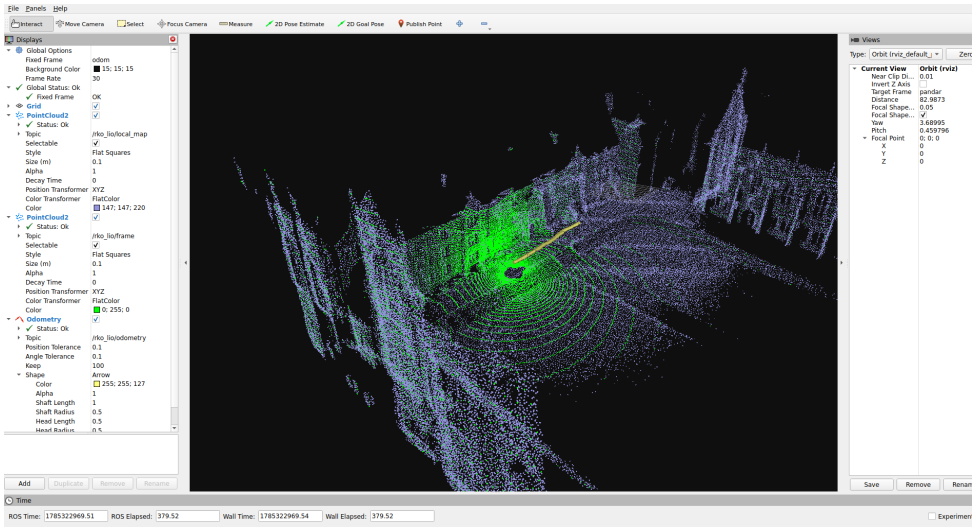


Figura 6: Visualización de la reconstrucción de la parte inicial de la secuencia en *Keble College* del OSD utilizando RKO-LIO

4.2. Guía de Ejecución de RKO-LIO con Oxford Spires Dataset

Para la ejecución de RKO-LIO con el OSD, se proporciona una de las secuencias del *Keble College* en el OSD, así como un archivo con la configuración concreta a utilizar para este dataset. La secuencia está en forma de *rosbag* y contiene escaneos LiDAR, imágenes de tres cámaras distintas y observaciones inerciales (IMU). En el siguiente enlace se podrán encontrar los materiales mencionados:

https://drive.google.com/drive/folders/1LaoKVN74P8LwPM8vF6ZdRd_l6eM6QsAc?usp=drive_link,

donde la secuencia del *dataset* es el directorio `1710255615_2024-03-12-15-00-16` y el archivo de configuración es `oxford_spires.yaml`. Ambos elementos deberán copiarse en la carpeta compartida con el contenedor: `~/ros2_ws`

A continuación, se deberán abrir dos terminales dentro del contenedor Docker y ejecutar los siguientes pasos:

Paso 1: Lanzamiento del Nodo LIO utilizando el archivo de configuración proporcionado:

```
# Lanzar el estimador rko_lio indicando la ruta al fichero de yaml
ros2 launch rko_lio odometry.launch.py initialization_phase:=false rviz
    :=true config_file:=/home/developer/ros2_ws/oxford_spires.yaml
    use_sim_time:=true
```

Nota: El nodo se inicializará y lanzará un aviso (Warning) informando que se encuentra a la espera del primer mensaje del LiDAR para calibrar y fijar el marco del mapa de odometría.

En este punto se abrirá una ventana de *RViz2* vacía. Hasta que no se empiece a reproducir el *rosbag*, no aparecerá nada, ya que el proceso está a la espera.

Paso 2: Reproducción del Dataset Real En la segunda terminal, se va a proceder a reproducir el archivo de datos, es decir, el *rosbag*. Hay que tener en cuenta que el terminal debe estar en el directorio `~/ros2_ws`:

```
# Reproducir el Rosbag
ros2 bag play 1710255615_2024-03-12-15-00-16/ --clock
```

Al ejecutar este comando, el estimador RKO-LIO sincronizará temporalmente los mensajes, inicializará el mapa de odometría con el primer escaneo y comenzará a generar de forma progresiva e iterativa la nube de puntos densa acumulada del entorno urbano del *Keble College* de Oxford, como se muestra en la Figura 6.

5. Bloque 4 - Manipulación Robótica (MoveIt 2)

5.1. Contexto y Objetivos del Bloque

La manipulación robótica introduce al alumno en el control de cadenas cinemáticas abiertas articuladas. A diferencia de la robótica móvil de par cinemático plano, los brazos industriales operan en el espacio tridimensional ($\mathbb{R}^3$), lo que requiere herramientas matemáticas capaces de resolver la cinemática directa (mapeo de ángulos de articulación a coordenadas cartesianas) y la cinemática inversa (cálculo de los ángulos necesarios para alcanzar un punto del espacio).

En este bloque volveremos al simulador para utilizar el brazo robótico **Universal Robots UR3e**, un cobot de 6 grados de libertad ampliamente utilizado en entornos industriales. El ecosistema de software se compone de tres pilares integrados:

1. **Gazebo Harmonic:** Proporciona el motor físico y de colisiones donde está anclado el brazo virtual.
2. **`ros2_control`:** Actúa como la capa de abstracción del hardware (los "músculos" del robot), gestionando los controladores de trayectoria de las articulaciones (*Joint Trajectory Controller*).
3. **MoveIt 2 (`move_group`):** Actúa como el motor de planificación y cinemática (el "cerebro"), calculando trayectorias libres de colisiones mediante algoritmos de muestreo espacial (OMPL).

5.2. Guía de Ejecución y Control Interactivo

Gracias a la preinstalación integrada de la pila de Universal Robots, el alumnado puede lanzar todo el ecosistema (física, cinemática y visualización gráfica) mediante un único script de *launch*.

Paso 1: Lanzamiento de la Simulación Unificada Abrir un terminal dentro del contenedor Docker (siguiendo el flujo del Bloque 1) y ejecutar el siguiente comando para cargar el brazo UR3e:

```
# Lanzar Gazebo Harmonic + MoveIt 2 + RViz2 configurado para el UR3e
ros2 launch ur_simulation_gz ur_sim_moveit.launch.py ur_type:=ur3e
    use_sim_time:=true
```

En este punto se podrá visualizar la simulación del brazo robótico en *Gazebo* como se muestra en la Figura 7. Además, también se abrirá *RViz*, lo que permitirá la interacción con el brazo, como veremos en el siguiente paso.

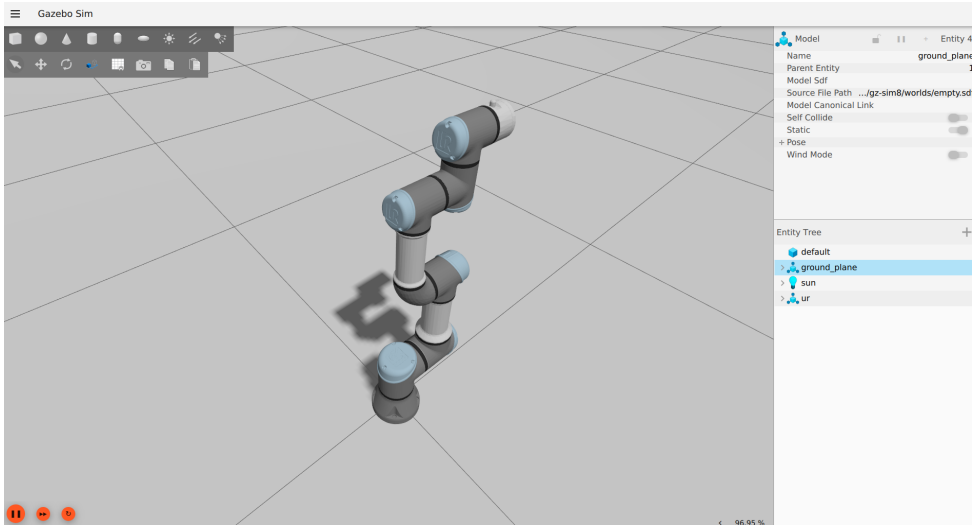


Figura 7: Visualización del brazo robótico en Gazebo.

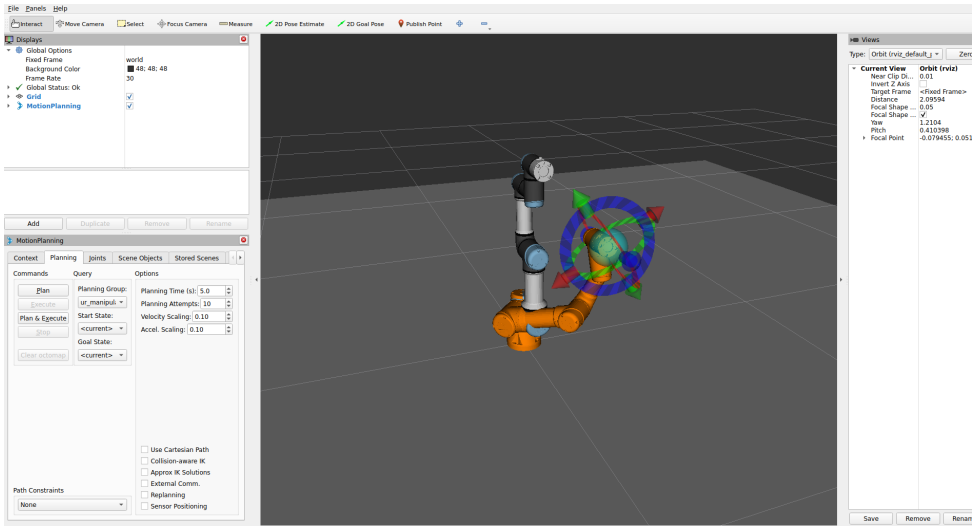


Figura 8: Visualización del brazo robótico en *RViz*.

Paso 2: Operación en el Espacio de Trabajo desde RViz2 Al arrancar el entorno, se abrirán las interfaces de Gazebo y RViz2. Para gobernar el brazo de forma interactiva, se deben seguir los siguientes pasos:

1. En la pantalla 3D, se puede observar el brazo de color **naranja**, que representa el estado objetivo (*Goal State*). En su extremo (*End-Effector*), aparecerá un marcador interactivo en forma de esfera tridimensional con flechas de colores (ver Figura 8).

2. Pinchar y arrastrar las flechas con el ratón para desplazar el extremo del robot a una pose cartesiana deseada. Si se intenta llevarlo fuera de su alcance físico o a una singularidad matemática, la esfera saltará de vuelta a su posición original.

3. A continuación, hay que dirigirse al panel inferior izquierdo (*Motion Planning*), seleccionar la pestaña **Planning** y hacer clic en el botón **Plan and Execute**. El robot real de Gazebo ejecutará la trayectoria calculada de forma fluida.

6. Entrega y Criterios de Evaluación

6.1. Estructura de la Memoria Técnica Obligatoria

El trabajo desarrollado en los **Bloques 2, 3 y 4** se documentará en una única memoria técnica por equipo (formato PDF). Para cada uno de los bloques citados, se exigirá de manera obligatoria el análisis e introspección del pipeline de software. El alumnado deberá interrogar al middleware utilizando las herramientas CLI aprendidas para diseccionar la arquitectura lógica que sustenta cada simulación/experimento. Cada bloque debe incluir:

- **Capturas y Diagramas del Grafo:** El grafo dinámico de nodos obtenido mediante la herramienta gráfica `ros2 run rqt_graph rqt_graph`.
- **Mapeo de Interfaces:** Una tabla detallada que liste los nodos principales activos, especificando de forma explícita qué tópicos (*topics*) publican/suscriben, qué servicios ofrecen y qué acciones gobiernan el flujo.
- **Explicación del Pipeline:** Una descripción justificada por el alumno sobre el funcionamiento del sistema tanto a alto nivel (propósito del bloque) como a nivel de nodo individual. Se debe responder con claridad a: ¿cómo fluye la información matemática y sensorial desde que se genera la orden o lectura del sensor hasta que se traduce en un movimiento físico del robot?
- **Análisis e Interpretación del Árbol de Transformaciones (TF):** Comprender las relaciones geométricas espaciales es un requisito indispensable en robótica. Para los bloques 2, 3 y 4, el alumnado deberá extraer el árbol de transformaciones espaciales en formato PDF empleando los comandos:

```
# Generar el diagrama del arbol de transformaciones geometricas
ros2 run tf2_tools view_frames

# Consultar mensajes de transformaciones
ros2 topic echo /tf
```

6.2. Tareas de Programación (Opcionales para Subir Nota)

Para aquellos equipos que aspiren a alcanzar una calificación extra, se proponen dos tareas de desarrollo de código:

- Desarrollar un script para enviar una pose deseada mediante código Python para navegación autónoma.
- Desarrollar un script para enviar una pose deseada mediante código Python para el brazo robótico.

Ambos scripts deberán implementarse en Python dentro del espacio de trabajo persistente del contenedor (`~/ros2_ws`) y llamarse mediante comandos de consola sin recurrir a clics manuales en RViz2.

6.3. Criterios de Evaluación

La evaluación de esta práctica no se realizará únicamente mediante el informe, sino que además se realizarán consultas presenciales e individuales en el aula de laboratorio, estructurándose en dos fases:

1. **Demostración en tiempo real:** Cuando se finaliza cada bloque, el equipo de prácticas deberá tener desplegado y ejecutándose en sus puestos de trabajo el contenedor Docker con la simulación del bloque correspondiente activa y respondiendo a comandos. No se evaluará ninguna sección basándose únicamente en capturas del informe.

2. **Preguntas a nivel individual:** Con el entorno simulado en marcha, el profesor realizará preguntas orales individuales e independientes a cada uno/a de los/as integrantes del equipo de prácticas.

La calificación final del bloque resultará de la media ponderada entre la calidad analítica de la memoria entregada, la correcta ejecución del código (incluyendo los puntos opcionales) y, con un peso crítico, la soltura y madurez demostradas por el alumno durante la ronda de preguntas orales.